

Middleware for Distributed Applications in a LoRa Mesh Network

JOAN MIQUEL SOLÉ, Universitat Politècnica de Catalunya, Barcelona, Spain

ROGER PUEYO CENTELLES, Universitat Politècnica de Catalunya, Barcelona, Spain

FELIX FREITAG, Universitat Politècnica de Catalunya, Barcelona, Spain

ROC MESEGUER, Universitat Politècnica de Catalunya, Barcelona, Spain

ROGER BAIG, Universitat Politècnica de Catalunya, Barcelona, Spain

Recently, LoRa mesh networks have gained increasing interest as a communication layer for sending data between IoT nodes. However, the network service of the firmware on the microcontroller-based nodes is typically limited to sending and receiving LoRa packets through the LoRa radio. Therefore, the packet processing by the node has to be done using an application-specific implementation. In this article, we present the design and implementation of a middleware that facilitates the development and operation of multiple distributed applications on LoRa mesh network nodes. The components we propose leverage the routing capacity of a LoRa mesh network enabled by the LoRaMesher library and provide a service for applications to send and receive messages from each other. Running several applications concurrently is also supported. We experiment with the middleware implemented in the node firmware with distributed applications that span from the LoRa mesh network to the Internet over MQTT. Our results show the support of bidirectional application-level communication, which can be used to build cross-network distributed applications that integrate services on LoRa mesh network nodes.

CCS Concepts: • **Networks** → **Network services**; • **Hardware** → **Wireless devices**;

Additional Key Words and Phrases: LoRa, mesh network, IoT, distributed application

ACM Reference Format:

Joan Miquel Solé, Roger Pueyo Centelles, Felix Freitag, Roc Meseguer, and Roger Baig. 2025. Middleware for Distributed Applications in a LoRa Mesh Network. *ACM Trans. Embedd. Comput. Syst.* 24, 4, Article 60 (July 2025), 26 pages. <https://doi.org/10.1145/3747295>

This work was funded by the Spanish Ministerio de Ciencia Innovación y Universidades through the grant PDC2023-145809-I00/AEI/ 10.13039/501100011033; and PID2023-146066OB-I00 (AEI 2023 Knowledge Generation Projects); the Recovery and Resilience Mechanism of the European Union, and by the European Union NextGenerationEU; the Generalitat de Catalunya through the grant 2021-SGR01059.

Authors' Contact Information: Joan Miquel Solé, Universitat Politècnica de Catalunya, Barcelona, Spain; e-mail: joan.miquel.sole@estudiantat.upc.edu; Roger Pueyo Centelles, Universitat Politècnica de Catalunya, Barcelona, Spain; e-mail: roger.pueyo@upc.edu; Felix Freitag, Universitat Politècnica de Catalunya, Barcelona, Spain; e-mail: felix.freitag@upc.edu; Roc Meseguer, Universitat Politècnica de Catalunya, Barcelona, Spain; e-mail: roc.meseguer@upc.edu; Roger Baig, Universitat Politècnica de Catalunya, Barcelona, Spain; e-mail: roger.baig@upc.edu.



This work is licensed under a Creative Commons Attribution 4.0 International License.

© 2025 Copyright held by the owner/author(s).

ACM 1539-9087/2025/07-ART60

<https://doi.org/10.1145/3747295>

1 Introduction

LoRa mesh networks have emerged as a complementary usage of LoRa communication technology beyond LoRaWAN [31]. In mesh networks, **Internet of Things (IoT)** nodes can send messages over LoRa to each other. The multi-hop capacity in LoRa mesh networks extends the geographic scale of the communication between any two LoRa nodes. The extension of the basic network service of LoRa mesh networks could provide a technology that allows the building of new types of distributed applications hosted on the IoT nodes [26].

LoRa is nowadays mainly used in LoRaWAN-based applications [19]. These applications are hosted in the cloud, enabling a path for data from the sensor nodes to the cloud-based service. LoRa nodes acquire sensor values, which are then sent to a gateway node with an Internet connection. The data is then forwarded to cloud-based services for analysis and visualization. The data transfer is one-way, from the sensor nodes to the cloud. Although LoRaWAN has a downlink capacity, it is limited compared to the uplink. It is minimal and rarely used.

Initial application cases for LoRa mesh networks have already been shown [1]. Meshtastic is a practical application that uses a LoRa mesh network as a communication substrate. In Meshtastic, a smartphone as the end-user device connects over Bluetooth to a tiny LoRa transceiver node that is part of the Meshtastic LoRa mesh network. Users can send short text messages to other users connected to the network. Meshtastic does not use gateway nodes with an Internet connection.

Only a few open-source libraries are available to implement LoRa mesh networks. One of the libraries is RadioHead [21], which in [8] was used to implement and evaluate a small-scale LoRa mesh network. Another is LoRaMesher [28],¹ which we use in this work. LoRaMesher enables routing in the LoRa mesh network by applying a proactive distance-vector protocol. The library is implemented with FreeRTOS and compiles for ESP32-based microcontroller boards. Its basic network service sends LoRa packets from a source node over a routed network to a destination node. An advanced service is also available that provides reliability by the destination node confirming the packet reception to the source node. In both network services of LoRaMesher, however, a node cannot operate more than one application. The proposed middleware overcomes this limitation and enables running multiple networked applications at the same node.

The scenario we address in this work is that IoT nodes that are part of a LoRa mesh network can operate multiple distributed applications. These applications can be within the LoRa mesh network or span over to Internet hosts. We envision a middleware that manages the application networking support on a node on top of packet delivery that is provided between any two nodes by LoRaMesher.

In this article, we present a middleware called LoRaChat² that supports running multiple networked applications on top of the LoRaMesher library [28] in a LoRa mesh network. We implement the components in the LoRa mesh node firmware and demonstrate the running of distributed applications in experiments that are conducted on real IoT nodes.

The main contributions of this article are:

- Identification and design of middleware components to support distributed applications in LoRa mesh networks.
- Implementation of the middleware as an extension of the LoRaMesher library.
- Demonstration of the middleware operability with prototypes of distributed applications running on real IoT nodes.
- The middleware, the LoRaMesher library, and example applications are available as open-source in public git repositories.

¹<https://github.com/LoRaMesher/LoRaMesher>

²<https://github.com/Jaimi5/LoRaChat>

In the development of this work, several challenges have been addressed:

- (1) **Application identification:** When multiple distributed applications run on a node concurrently, each application needs to be identified when it communicates over the network. We introduce *application ports* that allows relating the payload of each LoRa packet with the corresponding applications. The use of application ports required designing a new packet header to contain the application-level data, and the components of the middleware to process it.
- (2) **Application multiplexing:** Each node has to run a component that manages the networking of the applications running on that node. The received and sent messages of multiple applications have to be processed at the node. When receiving a message from the network layer, the payload has to be demultiplexed to the destination application. When an application on the node creates a message, the obtained information, i.e., application source, application destination, node source, node destination, and data, has to be converted into an application payload that is delivered to the LoRaMesher network service. We designed a component called *MessageManager* that implements the message processing in the middleware.
- (3) **Cross-network applications:** To support distributed applications that are not only within the LoRa mesh network nodes but can span from the LoRa mesh network to other networks, the network destination needs to be identified. For instance, a gateway node, when it receives a LoRa packet as a destination node within the LoRa mesh network, needs to determine whether a received message has to be delivered to an application at that node or to the Internet. In the second case, the application payload has to be forwarded to the gateway's MQTT client. Our design introduces *message ports* that identify networks, i.e., LoRa mesh, Bluetooth, and WiFi networks to enable cross-network applications. Message ports allow the middleware on a node to determine the destination network.
- (4) **LoRaMesher as a network service:** The implementation of the middleware required to change the integration of the LoRaMesher code to support not only being a standalone software but also being a layer that provides a network service. For this, public interfaces to send and receive functions, besides other functions, were made accessible for the upper middleware layers.
- (5) **Usability:** It was important for the developed middleware to be easily extensible with third-party applications. For this, the design of the middleware contains as core components the generic *DataMessage* and the *MessageManager* class. This *DataMessage* allows to customize the data message formats for the specific application. The *MessageManager* is instantiated with the applications that it has to manage, which adds the flexibility to include third-party applications. Several sample applications are provided in the LoRaChat git repository to practically show an application implementation. For instance, the *Led* application, to actuate on request, and the *Sensor* applications, to operate periodically, can be reused by developers as a blueprint to implement their own applications.

This article is structured as follows: Section 2 provides a detailed review of the related work, looking into application-level aspects, multiple application support, prototypes, and commercial solutions, and compares these works with our solution. In Section 3, we introduce the concept of distributed applications in LoRa mesh networks. In Section 4, we explain the basics of the LoRaMesher library necessary to understand the design of the middleware. Section 5 goes into the details of the middleware design. We describe the new packet format, the *MessageManager* and its integration, and how the applications have been integrated into the task-based execution of the node's software. In Section 6, we evaluate the middleware software deployed on real IoT nodes. In the first set of experiments, we observe details of the performance of multiple and different types of applications running concurrently on IoT nodes. We then conduct a larger-scale deployment of

nodes with a longer-term duration to observe the scalability and robust operation of the middleware running on the nodes. Section 7 discusses the results of our work. Finally, in Section 8, we conclude the article and indicate future directions.

2 Related Work

2.1 Applications with LoRa Networking

Several survey articles on LoRa communication have been published in recent years. Their main focus is typically the technological and communication aspects of LoRa, as well as performance, networking, and limitations. The application layer, use cases, and lessons learned from real-world deployments are covered to some extent, but usually in less depth. In the survey of [29], applications are reviewed from the perspective of the types of applications that have been enabled, but without deeply analyzing how these applications are designed from an architectural point of view. In the survey of [19], the MAC layer and the application layer are studied in detail, but the work focuses on applications that use the LoRaWAN architecture. Unlike in LoRa mesh networks, however, in LoRaWAN the data is transmitted by a 1-hop link between the end node and the gateway, so there is no network for communication between end nodes. The survey of [23] reviews real system implementations and summarizes application cases, but the focus is again on applications that use the LoRaWAN architecture.

It can be concluded from these surveys that most existing works about applications with LoRa technology are centered on LoRaWAN. Applications that use LoRa mesh networks are not yet well covered. However, the application operation in LoRaWAN is different from that in LoRa mesh networks. In LoRaWAN, due to a lack of communication between end nodes, the application code on the IoT node can interface directly with the MAC layer. Furthermore, applications can be implemented ad hoc to fit the specific application requirements without an additional networking layer on top of LoRaWAN. Additionally, the applications typically involve tiny sensor-equipped end nodes that use the LoRa technology only to communicate their data to the Internet. While the surveys show that the requirements of many traditional monitoring applications can be satisfied with the LoRaWAN architecture, the concept of tiny end nodes that are mostly disconnected and only send from time to time data to cloud-based services, poses limits to reaching out with LoRa communication to new application areas, such as Edge AI [17]. For instance, the potential of inter-node networking could be exploited with nodes consisting of recent more powerful microcontrollers in machine learning-enabled embedded IoT boards [6].

Among the works that go beyond the architecture and applications of LoRaWAN, Ghosh et al. [14] proposed LoRaute, where a LoRa mesh network is conceived as a backhaul network with hosts and mobile nodes. The nodes are further classified into base station, home station, and mobile nodes. The distinction allows assigning different operation modes to the nodes, where the base station nodes are the ones that run permanently to provide a LoRa-based backbone network. The base station routing nodes request their neighbors for a list of their neighboring nodes. The obtained list forms the routing table of that node. The protocol implements an optional acknowledgment-based message-sending retry. A smartphone is used as the device which hosts the application. The smartphone connects over WiFi to a node of the LoRaute mesh network node. Like in our work, in [14] the hardware implementation of the nodes was done with ESP32-based boards. The proposed base station can be related to the routing nodes of LoRaMesher. As in our work, these nodes are considered to be permanently operational. The smartphone-based end-user implementation, also proposed in [30] and [1] is different from our work where our proposed middleware enables the node to host the application and the networking stack. Acknowledge-based message delivery is implemented which is similar to what our middleware provides leveraging LoRaMesher. In addition, our middleware enables reliable delivery of application payloads beyond the size of a LoRa packet

while the system in [14] was tested with data sizes of up to 180 B which can fit into a LoRa packet. A Python-based implementation is mentioned in [14], whose code unfortunately is not provided by the authors. Differently, our middleware and LoRaMesher are implemented in C++ and both are provided as open-source.

Vigil-Hayes et al. [30] presented LoRaX, a system that combines a low-performing but pervasive LoRa network with geographically limited access to high-performing Internet to connect underserved areas. For the interconnection, the authors introduce a proxy server that manages the data transfer between both networks. The architecture introduces compound repeaters to support bidirectional and multi-hop communication within the LoRa network. Some of the compound repeaters have Internet connectivity and they interact with an MQTT broker. The end user device of the user is a smartphone that hosts the application. The smartphone connects locally to an ESP32 board, which then has a serial connection to a local node of the LoRa network. Basically, LoRaX proposed the LoRa network as an access network for unconnected areas to reach a remote Internet access point. The work shows a real-world implementation of the compound repeater hardware and presents measurements taken from real deployments. Compared to our work, we share the concept of combining Internet services with a LoRa network. The implementation of our middleware also enables applications that run on both networks. Differently, the design seems to represent a monolithic system while our middleware is open to be used by multiple applications on a node. Unfortunately, the LoRaX software is not provided in an open code repository which could help to understand better many details of the implementation that the article does not sufficiently describe. Differently, the middleware we propose is open source and available to the community.

Arratia et al. [5] present a protocol design called AllLoRa for a LoRa mesh network based on a stop-and-wait **Automatic Repeat reQuest (ARQ)** scheme that forwards messages in a multi-hop network without a routing protocol. The authors demonstrate the system by solving a data transmission problem of buoy sensor nodes in a lagoon. The evaluation is centered on practical aspects of the IoT application case, such as the throughput with different SFs and energy consumption. In AllLoRa, only the source and sink nodes participate in the transmission of the chunks, and it is closed with a final ACK of the totality of the data. In our proposal, the source node initiates the transmission, and the ACK is per chunk. The AllLoRa packet format resembles the format of LoRaMesher focussing on node source and destination. In contrast, to enable multiple applications, our middleware extends the header with application-level information that allows to manage payloads to and from applications of a node.

Nakamura et al. [24] introduced the LoRaCTP protocol to support the reliable transfer of chunks of data over LoRa. The work proposes edge nodes consisting of an MQTT broker and a content forwarder that connects to the LoRa network. The LoRaCTP protocol runs the content forwarder and introduces a specific header format for LoRa packets to enable a stop-and-wait ARQ approach for reliable message transfer. The protocol was implemented with MicroPython. The evaluation was done with only two nodes and focused on testing a message transfer between content forwarders with different **Spreading Factors (SFs)** and message sizes. In comparison to that work, the reliable transfer of large messages is a network service which also LoRaMesher offers to applications, but differently, over a multi-hop network where the packet is routed to its destination. The work of [24] introduced an MQTT broker as part of the architecture but does not further elaborate on its integration. Differently, our middleware integrates an MQTT client and its usability is demonstrated in the experiments of Section 6 with distributed applications that span over to the Internet.

Moreno Cardenas et al. [9] presented a LoRa network as a low-cost communication system for messaging. The architecture proposes edge nodes called hubs which have a WiFi and LoRa interface. The end user connects from any PC or smartphone through WiFi to its local hub. The LoRa network connects that user with others connected to other hubs. Within the LoRa network,

stop-and-wait ARQ is used for reliable message transfer. The solution was implemented with MicroPython on ESP32-based boards. The work has some similarities in its concept with [30], while with the integration of Telegram and a Web-based interface, it also illustrates the applicability of the proposed system. Compared with our work, the authors present a LoRa communication system to transfer reliably a payload between LoRa nodes, where the application runs on devices external to the LoRa network. In our approach, the LoRa node runs the networking stack and the middleware to support multiple applications running on that node.

A common point that can be identified from the above works is that the application code for the LoRa node was developed ad hoc. In classic LoRaWAN applications, the LoRa node is typically limited to reading and sending out a sensor value. In the works beyond that address the interconnection of nodes beyond LoRaWAN, the LoRa network is conceived as a communication substrate, such as in [14, 30], and [5]. The design of the application can be considered monolithic and customized. Differently, the middleware that we propose provides an abstraction level of both the operating system and the communication. The networking capacity at a node can be conceived as a service, as introduced in Section 5, which the different applications running on that node could use.

2.2 Approaches for Multiple Applications

We have identified three approaches in the literature to enable multiple applications with a LoRa network. The approaches can be grouped into extending Internet-side components, by using external devices, and by a networking stack on the IoT nodes.

A first approach for enabling multiple applications using the data from LoRa nodes is to create different applications by extending the Internet-side components [12, 22]. The nodes in the LoRa network are used for data provision, but the sensor values, which may be diverse, once available at the gateway, may be multiplexed to different applications. Consequently, these applications share the same IoT devices and their sensors, as well as the LoRa communication. Differently, in the approach we propose, the node's middleware can run multiple applications simultaneously on the same IoT device. The novelty of our approach is that it enables the concurrent execution of various applications on the same LoRa device and within the LoRa mesh network, so increasing the number of applications in the LoRa network, and not by reusing sensor data for different applications in the Internet as followed in other works.

A second approach to running multiple applications over a LoRa network is using external hardware, such as a smartphone, desktop PC, or another microcontroller board to host the application. The application running on this hardware connects to the LoRa node via Bluetooth, WiFi, or UART. This approach uses the LoRa device only as an entry point to a communication network. Höchst et al. [16] designed and implemented a device-to-device messaging application applying this approach. Pueyo et al. [10] made a similar proposal that uses the LoRa network as a communication substrate for disaster and rescue scenarios. The previously mentioned Meshtastic application implements this approach too [1]. In LoRaute proposed by Ghosh et al. [14], the smartphone is also used as the device which hosts the application. The smartphone connects over WiFi or Bluetooth to a LoRa mesh network node where that node has two network interfaces. Instead of using a smartphone to host the application, another node architecture is to build a sensor node with two devices, such as proposed in [18] and [20]. In both works, the boards are interconnected by UART. In [18], a Nano 33 BLE Sense board is used for the machine learning function, and an ESP32 NodeMCU board with LoRa RFM95 transceiver module is used for the networking. The data amount transmitted from the node to the remote IoT gateway is rather small, focusing on the classification data. In [20], the Arduino Portenta board is used to conduct the machine learning task. Different from that approach, in our proposal, we use a single device and the middleware on the node enables the execution of

applications directly on that device with the LoRa radio. Our solution does not require an additional PC, smartphone, or microcontroller board where the application code runs.

A third approach to enabling multiple applications to use the LoRa network concurrently is to implement a networking stack, such as one similar or compatible with TCP/IP. Sakamoto et al. employed IP2LoRa to leverage the TCP/IP stack on the LoRa device [27]. IP2LoRa is an application that establishes an IP tunnel over LoRa, serving as a network interface on a Linux computer. This requires an extra computer to operate IP2LoRa and establish a connection with the LoRa device through a serial port. Essentially, IP2LoRa can be interpreted to treat the LoRa device solely as a modem, similar to [16], but with an enhanced network service. In contrast, our proposal places the network service on the IoT node and utilizes the middleware to facilitate the execution of multiple applications on the LoRa device itself. Unlike the IP2LoRa approach, our solution does not need an additional computer to run the networking stack.

Haubro et al. use the IETF standard TSCH and 6TiSCH approach to send UDP packets over LoRa links [15]. LoRa is used as the physical layer of the networking stack, to enable reliable networking that benefits from LoRa's long-range operation. The integration of the LoRa radio into the networking stack was implemented for the Contiki-NG operating system. Haubro et al. position the communication concurrency within the TCP/IP stack, delegating communication decisions to this layer. Differently, in our proposal, we use the LoRaMesher library [28] as a network service for routed packet delivery in the LoRa mesh network, and LoRaMesher uses RadioLib [4] to interface with the LoRa radio. Our proposal is a middleware to support the application layer of the node. It empowers application designers and developers to make decisions regarding communication configuration, including packet sizes, message scheduling from various applications, and other pertinent parameters. Placing this control in the hands of the application developer facilitates a flexible approach to customize the communication of each application, which can be within the LoRa mesh network or spread to Internet-based services.

In summary, although there have been works that extended the features of a networking stack on the base of LoRa communication, there are no works that have demonstrated multiple applications with different communication patterns running simultaneously on LoRa mesh network nodes as we do in this work. This underlines our proposal's novelty and value in the middleware field for LoRa technology.

2.3 Commercial and Community LoRa Mesh Network Solutions

A few commercial providers offer multi-hop LoRa network solutions. They envision several application cases, such as interconnecting sensors in remote areas or underground networks, as in [11, 13], argued by the difficulty of using other technical solutions or by a lower economic cost of using sensor nodes with multi-hop capacity in LoRa mesh networks instead of increasing the number of gateways.

NeoCortec offers the NeoMesh solution running with LoRa [2]. NeoMesh is presented as a time-synchronized routing protocol to control the communication between sensor nodes organized in a mesh topology. The solution is proprietary and is offered with pre-certified hardware. There is the option to use the provided hardware as routing nodes only and put the application on a separate board connected over UART, which is the same concept as in [18] and [20]. Then, their solution distributes the application and network layers on two boards. Differently, in our work, we run the applications, middleware, and LoRaMesher network service on a single board. We remark that if only LoRaMesher is flashed on a device then this node behaves as a router, as in NeoMesh.

Orama-systems offers another commercial solution for LoRa mesh networks [3]. Specifically, the company provides integrated software and hardware for LoRa multi-hop networks. The solution is specified for networks with up to nine hops and several thousands of nodes. Nodes are equipped

with two kinds of software stacks to maintain the mesh network. The message propagation uses flooding for the downlink and routing for the uplink. The company claims that its approach can significantly extend the reach of nodes compared to LoRaWAN. However, the specific technology is not disclosed, and no open-source code is available. There is no evidence that their solution can support running multiple applications on nodes, such as our work does. The path to integrate an application into their LoRa mesh solution is not clear, while in our work we have multiple examples as blueprints in the LoRaChat and LoRaMesher's open git repositories that help developers add new applications.

Possibly, the most consolidated community application nowadays that uses a LoRa mesh network is Meshtastic [1]. In Meshtastic, a smartphone as the end-user device connects over Bluetooth to a node with LoRa radio that is part of the Meshtastic mesh network. Users can send short text messages to other users connected to the network. Meshtastic is an example of an application that runs within a LoRa mesh network. It is an isolated network that does not use any gateway nodes with an Internet connection. The communication substrate that enables the LoRa mesh network is part of the specific Meshtastic application. Meshtastic has a high technical readiness level. It is operational and real usage of Meshtastic has been demonstrated. Meshtastic is focused on supporting its specific text-messaging application on the smartphone with a networking substrate. Differently, the innovation we contribute is a middleware as a new service on nodes that supports running multiple applications on a LoRa mesh network node. Compared to Meshtastic, LoRaMesher that started in 2021 is a younger project. However, the community has already awarded more than 190 stars to LoRaMesher's open-source repository. The Readme of LoRaMesher repository has examples with detailed steps for being reproduced by others. The LoRaChat repository contains several implemented example applications, such as *led* or *monitor*, that can be used as blueprints for new application developments.

3 System Model and Assumptions

We target distributed applications that can be of two types: all nodes of the distributed application are within the LoRa mesh network, or the application can span over the Internet and the LoRa mesh networks, where it has nodes that are hosted on the Internet and other nodes that are in the LoRa mesh network (Figure 1). Other works previously discussed in Section 2 such as [24] and [9], also consider this scenario.

Figure 1 illustrates the three types of communications used by these types of applications: communication between any two nodes within the LoRa mesh network, communication from a LoRa mesh network node with a host on the Internet, and communication from a host on the Internet with nodes in the LoRa mesh network. Gateways with network interfaces for both networks enable communication between hosts on the Internet and the LoRa mesh network in any direction. Nodes in the LoRa mesh network are typically permanently connected, where batteries charged by small solar panels enable 24/7 operation. The related works of [30] and [14] also consider permanently connected nodes in the LoRa mesh network, but the type of communication that their system supports is not fully demonstrated.

Several distributed applications may concurrently run in the LoRa mesh network and on the IoT nodes at the same time. The network service provided to applications by a middleware should be designed for this case. Limitations may be set by the restricted hardware resources of tiny IoT nodes. The related work has not shown the case of multiple networked applications running on a LoRa mesh network node but followed such a direction with other approaches as explained in Section 2.2.

Data sent between nodes will have an application identifier to help the network service at a node deliver the data packets to the specific application. Within the LoRa network, it is possible to have nodes that act as routers only and others that host multiple applications but are also routers at the

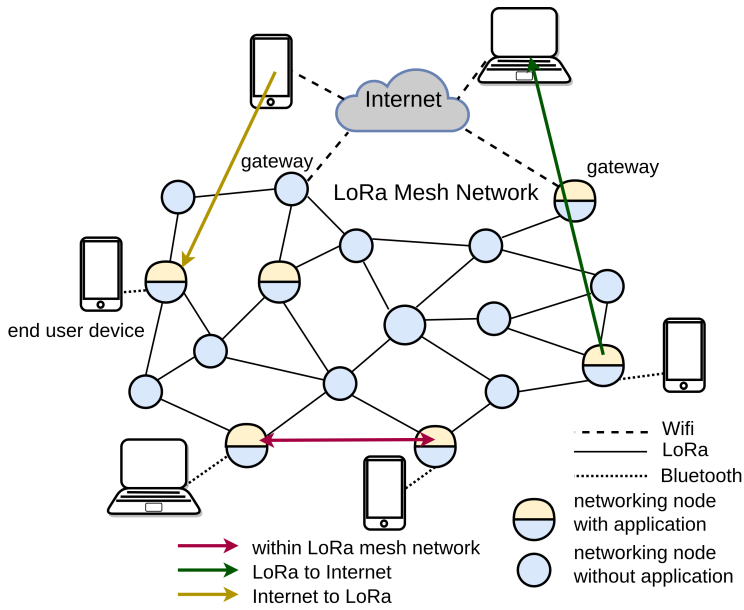


Fig. 1. LoRa mesh network with applications spanning over the Internet.

same time. The related work considered the LoRa mesh network as a communication substrate for only one application, such as [1]. Therefore, the packet headers that were proposed for LoRa mesh networks allow identifying the nodes, e.g., the destination of a packet. However, they do not contain information to distinguish multiple applications in the network, which is targeted within our scenario.

4 LoRaMesher Basics

The LoRaMesher library is a C++ implementation of a proactive distance-vector routing protocol for communication among LoRa nodes in a mesh network [28]. Its initial version represented a practical development of the design proposed in [25]. LoRaMesher uses FreeRTOS to provide tasks for packet reception, packet sending, packet processing, routing protocol, and application data handler.

For the interaction with the LoRa radio, LoRaMesher leverages RadioLib [4]. This library supports different radio chips of the hardware that we use and allows an easy configuration of LoRa parameters. Furthermore, LoRaMesher uses RadioLib to define an **Interrupt Service Routine (ISR)** which is executed every time a LoRa packet is detected. LoRaMesher compiles for embedded boards featuring an ESP32 microcontroller and a LoRa single-channel radio.

A LoRaMesher node sends two types of messages, namely, data messages and routing messages, which are identified at a node through the packet header. Routing messages are broadcasted with configurable periodicity. They are not forwarded by other nodes but help neighboring nodes update their routing table. Data messages are routed between any two nodes in the LoRa mesh network over multiple hops.

Queues are used at a node to enable the packet processing. There are three queues on each node for basic processing, namely, to store the received packets, the packets to be sent, and to forward packets to an application. For reliable messaging, additional queues are used to manage several data flows that may be active at the same time.

The applications that use LoRaMesher may have two main interactions with the library: sending messages to the destination and receiving messages from other nodes. Transparent to the application developer, LoRaMesher internally operates several tasks to receive and send LoRa packets at the network level [28].

To use the LoRaMesher library in an application for sending and receiving LoRa packets, the `LoRaMesher.h` file has to be included in the application code, for example in `main.cpp`. Then, in the application code, an instance of the `LoraMesher` class has to be created. The main configuration file of LoRaMesher is `BuildOptions.h`. It can be compiled with the default options or specific configurations can be customized. Among the configuration options is the role of the node, i.e., default or gateway, LoRa-related configurations (frequency band, SF, etc.), the `MAXPACKETSIZE` of the LoRa packet, timer settings, and routing packet settings.

If an application interacts with another instance over the network, a task for receiving and another for sending LoRa messages must be created at the application level. The *receiveData* task has to be implemented at the application level since the application has to implement how a received LoRa message is processed. The handler for the *ReceiveLoRaMessage* task is passed to LoRaMesher since the library will notify the Receive Task when a LoRa data packet for the node is received. The implementation of the *sendData* task enables to determine when the application sends messages, which can be, for instance, periodically.

4.1 Receiving Packets

To enable the reception of packets in the application code, the following steps are required:

- (1) Create a task for receiving packets in the application code and add the task handler to LoRaMesher.
- (2) LoRaMesher notifies the task when a packet for the application is in the queue. After receiving a notification, the application code creates a pointer of `AppPacket` type, to which then the address of the packet in the *Received User Packets* queue is given.
- (3) Process the data of the packet. For this, the applications use the `AppPacket`'s pointer to the payload, using the payload size obtained from the `AppPacket`.
- (4) Delete the pointer.

4.2 Sending Packets

LoRaMesher offers two options for sending packets to a destination node: one sends the data in a single LoRa packet without acknowledgment, and the other sends the packet with acknowledgment request from the receiver. The latter is a reliable transfer at the network level. LoRaMesher's function *createPacketAndSend* has to be called for the first option, sending a single packet without reliable transfer. The parameters are the destination address, a pointer to the payload, and the size of the payload. The data is accepted if it fits into the maximum permitted payload size.

For reliable transfer, LoRaMesher's *sendReliable* function must be called. It takes as parameters the destination address, a pointer to the content in `uint8_t` format, and the payload size in bytes. Note that the *sendReliable* function at the library level automatically splits the application data into data chunks that fit into the maximum size of a LoRa packet and. This operation is transparent to the application layer.

5 Middleware for Multi-Interface Application Communication with LoRaMesher

This section explains the design of the middleware. In our scenario, a LoRaMesher node should communicate with several applications and network interfaces, including LoRa, WiFi, and Bluetooth. We target a LoRa mesh network node that can host multiple applications and where LoRaMesher

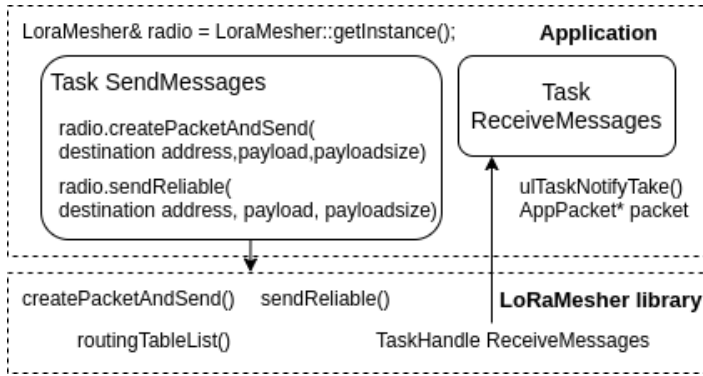


Fig. 2. Application and interaction with LoRaMesher library.

nodes with instances of these applications communicate with each other. Moreover, if the IoT device is equipped with a WiFi interface, then such a LoRaMesher node can also function as a gateway. The middleware and application instances should enable services running in a LoRa mesh network that are available to hosts on the Internet. To support this, an additional software codenamed *LoRaChat* on top of LoRaMesher was designed and implemented.

5.1 LoRaMesher Library Network Service to Applications

Figure 2 illustrates the network service the LoRaMesher library delivers to an application. For sending data from the application, we can observe the option of sending a small payload that fits in a single packet without reliable delivery and that of sending larger data through the library's reliable and large payload function. For receiving messages, the application has to implement a task, which is notified when a packet is ready to be read.

When sending a message, the destination must be specified. The application may know the address beforehand, or, through the public function *routingTableListCopy()*, the application can obtain the list of the other nodes in the network that are reachable from this node.

The basic sending and receiving of the LoRaMesher library and LoRa packets have the limitation that a node can run only one application. To solve this problem, the following sections describe the architecture of an additional layer designed to facilitate the exchange of packets among multiple applications on a single node. The reception of a LoRa packet is integrated into the *LoRaChat* code. Using this middleware, the data packet can be forwarded to different applications on the same host.

5.2 Multiplexing and Demultiplexing of Applications

We consider the case where a LoRaMesher node can host several different applications, each of which can be distributed. The network allows the application at one node to send data to or receive data from an instance of this application at another node.

To identify networked applications on a LoRaMesher node, we have introduced a set of codes to represent each application. In total, 256 codes (1 byte) are available. Table 1 shows some applications that have received codes. An example is a monitoring application, *SensorApp*, for which typically a Web server of this application is on the Internet. In such an application, the IoT node sends sensor data over the LoRa network to an *MQTTApp* application, implemented by an MQTT client on the gateway node, which sends the data to an MQTT broker on the Internet. It is possible to add a new application. For this, the codes of the table are extended with the new code given to that application.

Table 1. Example of Application Port Codes

Application	Code (Port)
LoRaChat	1
WiFiApp	2
MQTTApp	3
SensorApp	4
LedApp	5

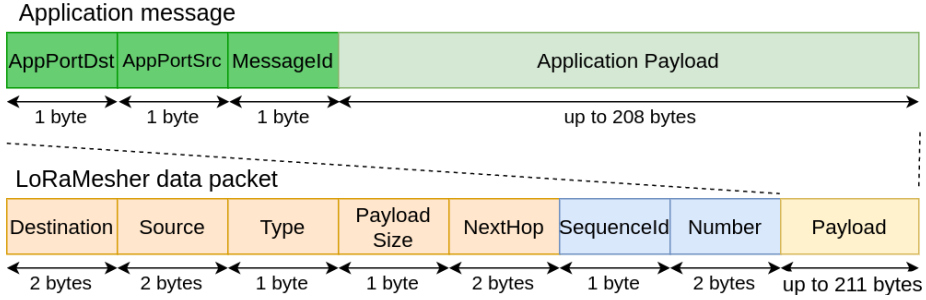


Fig. 3. LoRaMesher packet with application payload.

The applications shown in Table 1 are defined in the file `dataMessage.h`, where the applications are identified with their *appPort*. When multiple applications run on a network, the *appPort* identifier allows the packets received at the destination node to be delivered to the specific application. With this identifier, a generic data message header is created, containing an identifier for the message, the application destination port and source port, the message size in bytes, and the sender and receiver's source and destination address within the LoRa mesh network.

5.3 Packet Format

The application message is contained in the payload field of the LoRaMesher packet for reliable and large payloads (Figure 3). The application message consists of a header and the application payload. The header contains the application ports in the fields *AppPortDst* and *AppPortSrc*, used to identify the sending and receiving application, respectively; see Section 5.2. The *MessageId* field is an identifier commonly used in most applications to distinguish between messages. Additionally, it can be used to avoid duplication.

5.4 Message Ports

The file `dataMessage.h` contains the definition of message ports (Table 2). Message ports were introduced to forward packets at a LoRaMesher node between the node's communication interfaces. The message ports relate to the network interfaces available in the hardware. Two ports were created to ensure the node's WiFi availability. The *WiFiPort* addresses the sending of messages by HTTP and implies starting an HTTP client task after enabling the WiFi connection. The *MqttPort* connects the node to a WiFi network and starts an MQTT client at the node to publish and subscribe to the topics of an MQTT broker. The *BluetoothPort* enables sending and receiving messages over Bluetooth. With the *LoRaMeshPort*, messages can be sent over LoRa.

Table 2. Message Ports

Message Ports	Code
LoRaMeshPort	1
BluetoothPort	2
WiFiPort	3
MqttPort	4

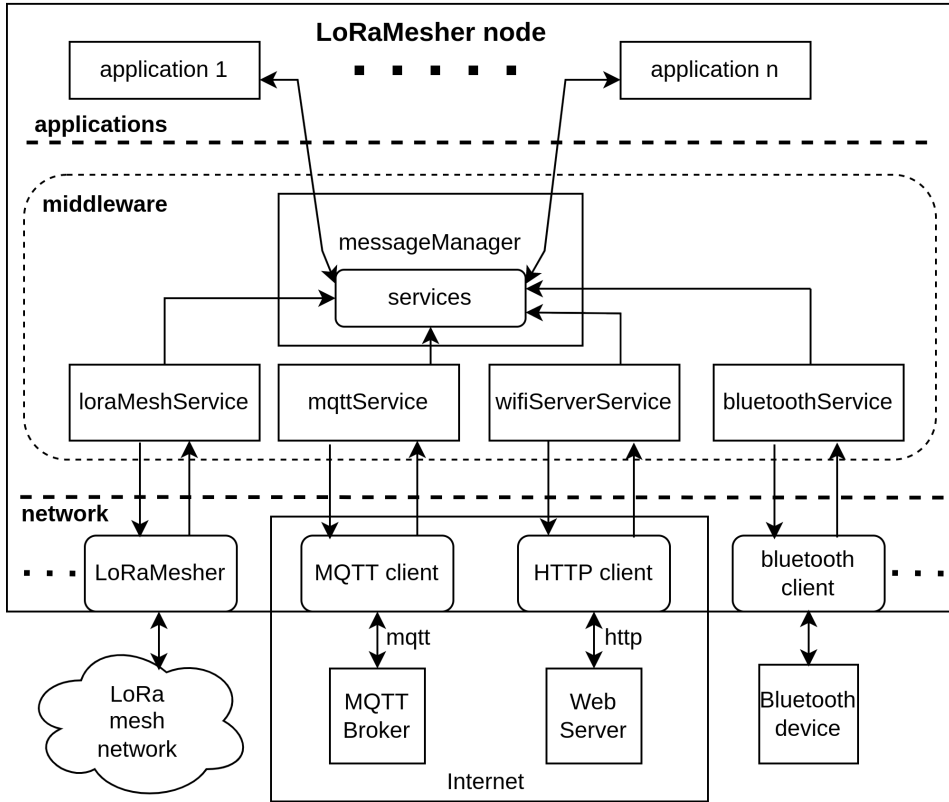


Fig. 4. Service components for message ports are registered at the message manager.

5.5 Message Manager

The *MessageManager*, implemented in the file `messageManager.cpp`, is a software component that organizes the delivery of application messages to the different destinations on a node. These destinations are the applications on the node and the message ports.

Figure 4 shows the position of the *MessageManager* within a layered view of the *LoRaMesher* software. The message ports are available through their service at the node's initialization. Each enabled service and application is registered in the *MessageManager*'s service vector. Also, the currently implemented *LoRaMesher* node connectors to three networks, that is, the *LoRaMesh* network, the *Internet* (through the MQTT and HTTP protocol), and *Bluetooth* are shown. From the architectural point of view, an additional connector could be added by implementing the corresponding service component and client.

An application is registered in the middleware by enabling the application in the `config.h` file of LoRaChat. As described in Section 5.2, each application requires an application port to be defined in the file `dataMessage.h`. For all enabled applications, an instance is created at board startup within the `setup()` function of the `main.cpp` file, and the application is added to the message manager's service vector. After finishing the setup, the identifiers of the enabled applications are contained in the service vector. The design does not limit the number of applications that can be enabled on a node. Each application, however, has specific resource requirements, which depend on the application type, amount of dynamic data, and implementation. To determine the feasibility of running multiple applications on a node in practice, the application type, usage pattern, and memory requirement of each enabled application should be taken into account.

An example of the MessageManager operation is the following: When receiving a LoRa message, first, at the level of the LoRaMesher library, it is checked whether the node's LoRaMesher address matches with the address of the destination field of the LoRaMesher packet. When it is confirmed that the LoRa packet has arrived at its destination node, the library delivers the packet's payload to the MessageManager component. Based on the content of the `AppPortDest` and `AppPortSrc` fields in the application message, the MessageManager determines if the application payload is delivered to an application hosted at this node or if it is sent to one of the ports, e.g., MQTT, at the device.

5.6 Task Interactions

Application- and network-level message processing in a LoRaMesher node is conducted by a set of tasks. Figure 5 shows these tasks and illustrates some activities. These tasks have different priorities. Multiplexing of data messages received by a LoRaMesher node to applications is achieved by the *User Receive Task*. The received data message is passed to the MessageManager, and the `AppPortDest` is analyzed to match the destination with the corresponding service from the service vector. The MessageManager invokes the message processing function of the specific service. As mentioned in Section 5.5, each application is registered at the initialization of the node as a service. The specific service then performs the action extracted from the received message content.

Different options exist for the action that is performed by the application. For instance, after processing, the action could include sending a message back to the network in response to the received message or setting a value at the node. In Figure 5, the latter case is shown where no message is sent. However, an application may also need to send some data periodically. This application will be implemented as a service containing a task to operate regularly, as illustrated by the tasks App 1 to App n in Figure 5. The proposed middleware components design allows several application services to run concurrently on a node.

Depending on the type of application and the usage pattern of each registered application at the middleware, different requirements arise regarding the node's resource availability, e.g., the microcontrollers' memory and CPU, and the node's energy budget. Due to the low data rate of a LoRa network, communication-intensive applications can affect the number of applications that can run concurrently. If the reliable transfer of LoRaMesher is chosen, see Section 4.2, the number of messages stored in the queues and hence the memory consumption of this application can increase in the case of packet losses.

5.7 Gateway Node and Role Service

As seen in Figure 1, gateway nodes are LoRa mesh network nodes connected to the Internet. At startup, a node tries to connect to the WiFi network that is configured in the node's software, specifically in `config.h`. After a successful connection, the node obtains an IP address, has a TCP/IP network service, and is also part of the LoRa mesh network.

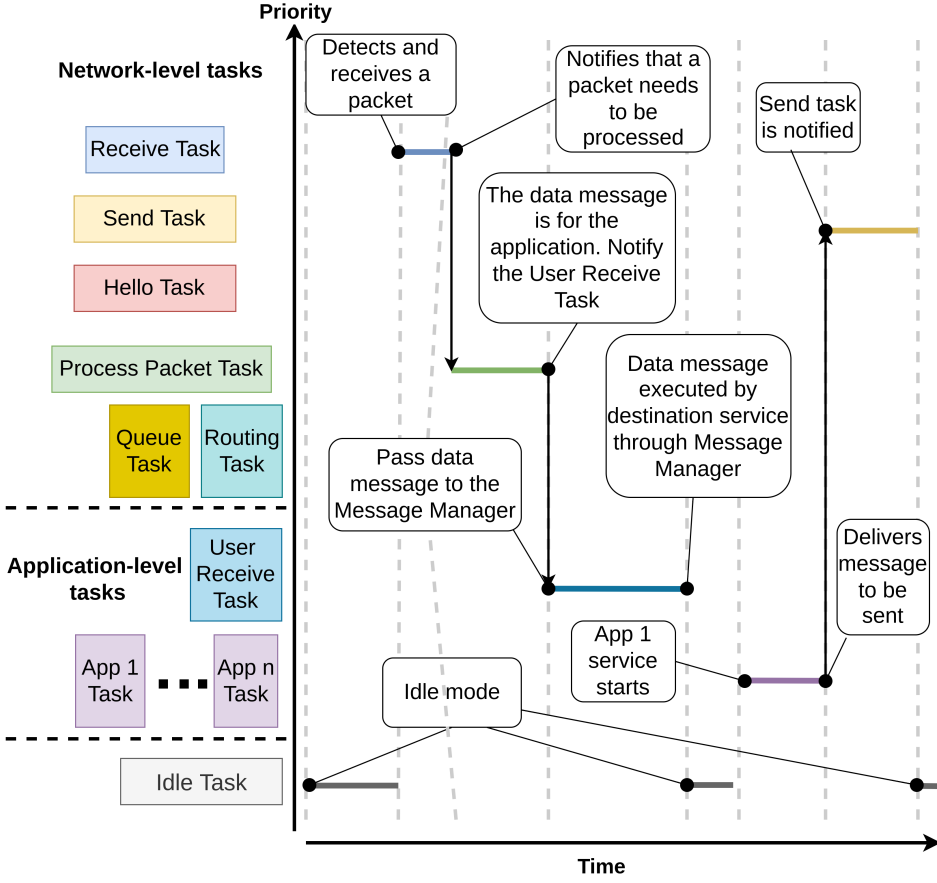


Fig. 5. Task on a LoRaMesher node combining networking service and applications.

A gateway node's Internet connection distinguishes it from other remote nodes in the LoRa mesh network. The gateway nodes enable applications to be distributed across both networks. The question arises of how the gateway capacity can be made known to the other nodes in the LoRa mesh network. An option could be to provide a service that allows nodes in LoRa mesh networks to discover gateway nodes. Another option, and the one that was chosen for our design, is to piggyback this information in the routing table exchanges. Therefore, when the nodes exchange their routing table, an additional 1-byte information is exchanged, which contains the role of the node, such as whether the node is a gateway. Applications on remote LoRa mesh nodes that aim to send messages to an Internet service can choose the nearest gateway node as the destination node from the identification of the gateway nodes in their routing table.

5.8 Integration of Applications Spanning Over the LoRa Mesh Network and the Internet

The gateway node can enable bidirectional traffic between the LoRa mesh network and the Internet. On the Internet side, HTTP and MQTT are popular protocols running on top of TCP.

With regard to HTTP, for bidirectional traffic with HTTP, both a client and server need to be implemented at the node. While in higher-end computing devices, having both components is not a problem, an HTTP server in the IoT node appeared to be a heavy task that is difficult to schedule

in compatibility with the task priorities of the network service, where specifically the Receive Task must have a high priority to not lose LoRa messages from the radio (Figure 5). Nevertheless, an HTTP client was implemented to support the unidirectional data message sent from the gateway node to an Internet-based host, as required by some monitoring applications. However, MQTT has been chosen as the main way to connect applications in the LoRa mesh network with the Internet. An MQTT client at the LoRaMesher gateway node can send and receive messages from an MQTT broker through publish/subscribe.

Publishers and subscribers connect through topics at the MQTT broker. Therefore, topics must be created on the MQTT broker. In our design, the gateway creates the topics to which it publishes. Specifically, for each LoRaMesher node that sends data to the Internet, a topic is created at the MQTT broker. This leads to more than one topic if several nodes in the LoRa mesh network send data to the Internet. Therefore, an Internet host obtains the data of a specific node by subscribing to that topic. If several applications run on a specific LoRaMesher node, all the data it sends will be received by the Internet host through that node's topic. Messages are in a JSON format and from the analysis of the message, the MQTT subscriber can associate the content with the corresponding application.

For receiving messages from the Internet, the gateway node subscribes to *one* topic of the MQTT broker. The topic name is identified by the gateway's LoRaMesher address. All data messages that Internet hosts send to any node in the LoRa mesh network are published on this topic. To process the MQTT message at the LoRaMesher gateway node, the MQTT message payload has to follow a pre-defined JSON format. It must include the information for three fields for a LoRaMesher packet, namely, *AppPortDst*, *AppPortSrc*, and *Destination* (Figure 3). Once the MQTT subscriber of a LoRaMesher node receives the MQTT message, it can transform it into a LoRaMesher packet and send it to its destination in the LoRa mesh network.

6 Evaluation

6.1 Experimental Setup

The objective of the experimentation is to demonstrate the middleware by running distributed applications that use both LoRa mesh network nodes and Internet hosts. Figure 6 shows the setup for the experiments of Sections 6.2, 6.3, and 6.4. On the side of the Internet, we run an MQTT broker and an MQTT client on a PC. The LoRaMesher node acting as a gateway is also connected to the Internet and runs an MQTT client. On the side of the LoRa mesh network, we have the three nodes with IDs DF44, DD40, and DD3C. The nodes are three T-Beam boards with ESP32 microcontrollers. We flashed the code of the LoRaMesher library, the middleware implementation, and applications as the firmware on the boards. The applications used the reliable transfer mode of LoRaMesher which was introduced in Section 4.2. The gateway node DF44 is a full LoRaMesher node running the same code as the nodes DD40 and DD3C. However, it acts as a gateway because it can connect to a WiFi network. The routing tables of the nodes correctly identify DF44 as a gateway node.

6.2 LoRa Mesh Network to Internet Communication with a Monitoring Application

This experiment assesses the communication of data from nodes in the LoRa mesh network to the Internet through an MQTT broker. The chosen scenario is a monitoring application whose IoT nodes regularly send sensor data. We experiment with three nodes in the LoRa mesh network (Figure 6). All three send sensor data every 90 seconds. Node DF44 has a WiFi connection and also acts as a gateway. Hence, node DF44 sends its sensor data and also forwards the sensor data received from nodes DD3C and DD40 to its MQTT client.

Figure 7 shows the arrival time of the monitoring data on an MQTT client subscriber on the Internet for an experiment duration of 20 minutes. It can be seen that the data from the gateway

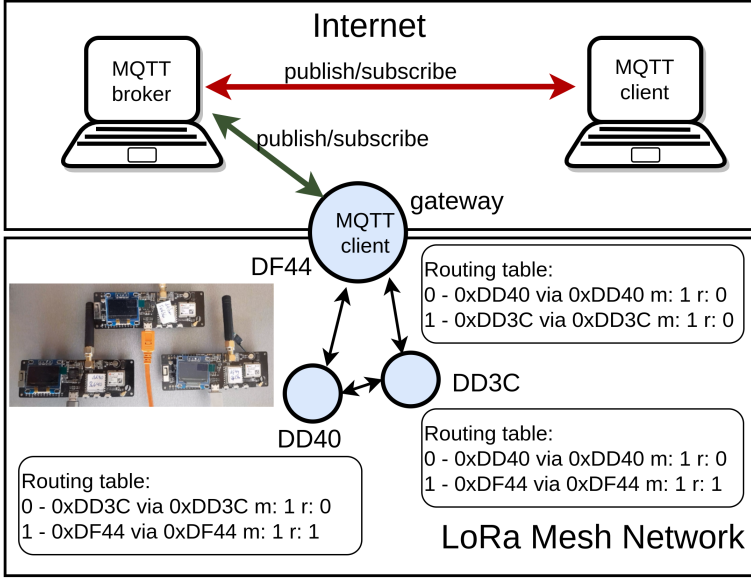


Fig. 6. Topology for the experimentation with LoRaMesher nodes DD40 to DD3C connected to the gateway node DF44 with Internet connection. Routing table entries with m: metric and r: role are shown.

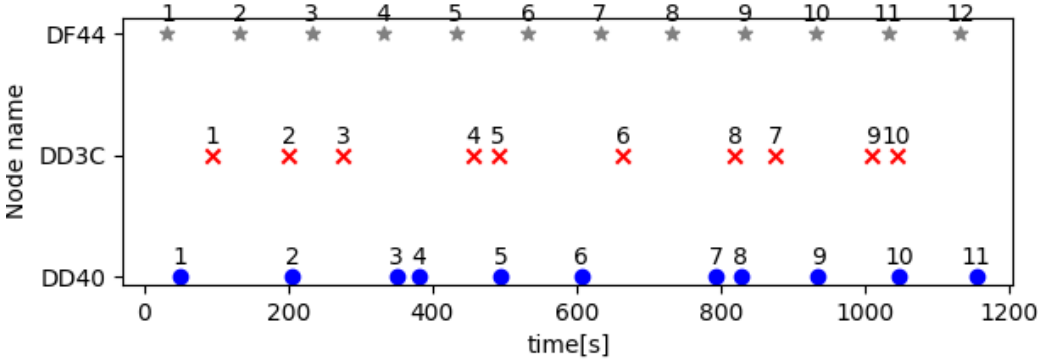


Fig. 7. Reception of monitoring data messages from three nodes in the LoRa mesh network at an MQTT subscriber in the Internet. The numbers indicate the ID of the messages sent from the corresponding node.

node DF44 are received at regular intervals at the MQTT subscriber. Differently, the data from nodes DD3C and DD40 is received in less regular intervals. This is because both these nodes communicate their data over the LoRa mesh network and packets can get lost. Specifically, collisions of the packets from nodes DD3C and DD40 can cause the packets not to arrive correctly at the gateway. These collisions can happen due to an overlapping sending of data packets from nodes DD3C and DD40 or from data packet collisions with a routing packet sent from any of the three nodes (the packets for exchanging the routing tables are sent every 30 s). If the gateway, the destination of packets from the nodes DD3C and DD40, does not confirm packet reception before timeout, the reliable messaging mode used in the experiments re-sends the data until the destination, node DF44, confirms the reception. As seen in the reception of messages 7 and 8 from node DD3C, reliable messaging can change the order of message arrival at the MQTT subscriber on the PC.

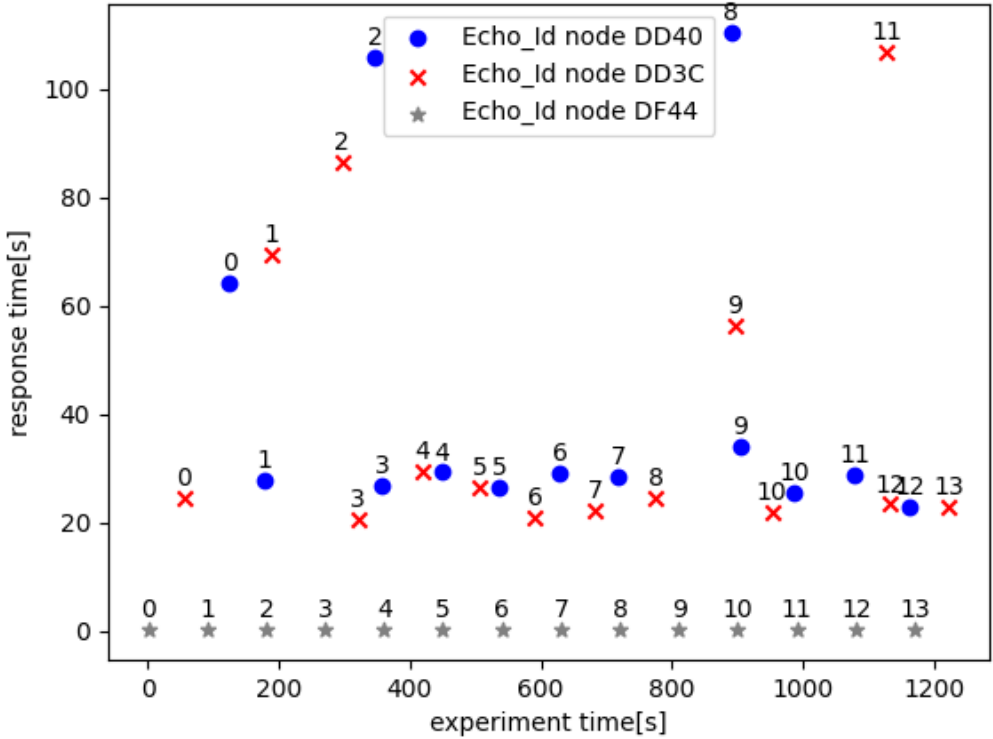


Fig. 8. Response time of echo messages sent from an MQTT client on the Internet over the MQTT broker to the three nodes in the LoRa mesh network. The node DF44 is the gateway. The numbers at the response indicate the ID of the echo message received from each node.

6.3 Bidirectional Internet to LoRa Mesh Network Communication with an Echo Application

In this experiment, the MQTT client at the PC illustrated in Figure 6 publishes every 30 seconds in round-robin order messages to three nodes in the LoRa mesh network. The message is received by the nodes and echoed back. We measure at the MQTT client at the PC the response time for each of the messages sent.

Figure 8 shows the response times for each message sent to the three nodes. The points in the figure correspond to one of the three nodes, as indicated in the legend. The x -axis value refers to the time of the experiment when the response was received. The y -axis value refers to the time it took until the response to the specific message was received. Node DF44 is the gateway, and the time corresponds to the reply between the publisher and subscriber of two nodes interconnected over WiFi and communicating over the MQTT broker. Differently, nodes DD3C and DD40 are nodes in the LoRa mesh network at one hop from the gateway. For some messages, the reply takes a long time, e.g., message 2 to node DD40 or message 11 to node DD3C. This increased response time implies that collisions took place and that the reliable payload protocol of LoRaMesher had re-sent packets. This can also lead to replies being received out-of-order, such as the replies to messages 2 and 3 of both nodes DD40 and DD3C. It can be observed that the reply time of nodes DD40 and DD3C is at least 20 seconds. During this time, at least four packets are exchanged for reliable messaging between the gateway and the node, where two are for the handshake, and two

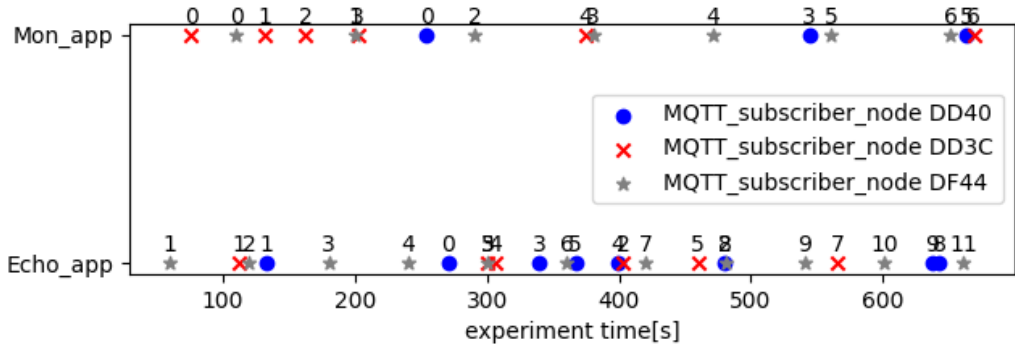


Fig. 9. Reception at the MQTT client on the PC of messages from the monitoring and echo application on the nodes DD40, DD3C, and DF44.

are for the data packet with acknowledgment. Another four packets are exchanged when the node sends back a data message to the gateway.

6.4 Multiple Applications Running at a LoRa Mesh Network Node

In this experiment, we study the capacity of the middleware to support running two applications at the same time. We use the monitoring and echo applications introduced in Sections 6.2 and 6.3, respectively. The MQTT client at the PC publishes every 30 seconds in round-robin order messages to three nodes in the LoRa mesh network, which are echoed by the nodes when they arrive. During the same experiment, the monitoring application at each node sends every 90 seconds a message to the MQTT broker, which the MQTT client subscriber of the PC reads.

Figure 9 shows the time of reception of the messages at the MQTT client subscriber on the PC. For this, the client has subscribed to three topics of the MQTT broker, each of them belonging to one of the nodes. The messages from the two applications on a node are received at the topic belonging to this node. The metadata of the messages allows the MQTT subscriber to classify each message according to its application, as shown in the figure. Furthermore, the figure shows the ID of the messages that have arrived. It can be seen that the messages of the gateway node DF44 are regularly obtained, while the messages from the nodes in the LoRa mesh network, i.e., nodes DD40 and DD3C, are obtained irregularly and out of order. As in the previous experiments of Sections 6.2 and 6.3, this demonstrates again the effect of the reliable message delivery mechanism active in the experiment. In this case, the experiment showed its operation for two applications running concurrently at the nodes.

6.5 Long-term and Large-scale Experiment

In this experiment, we analyze the operation of the middleware and LoRaMesher in a larger scale long-term experiment with a monitoring application that is run for 24 hours. The LoRa mesh network has 14 nodes. The nodes are ESP32 T-Beam boards as in the previous experiments presented in Sections 6.2, 6.3, and 6.4, and we run the MQTT broker with a subscriber on the Internet to obtain the messages from the 14 nodes.

Four of the nodes are configured as gateways and hence are connected to a WiFi network. The gateway nodes are identified with the numbers 56636 (DD3C), 59304 (E7A8), 12680 (3188), and 50884 (C6C4). The other ten nodes are connected only within the LoRa mesh network and are at one hop distance from all other nodes. Figure 10 shows a snapshot of the routing table at the gateway

```

I (6377488) LoRaMesher: Current routing table:
I (6377492) LoRaMesher: 0 - 142C via 142C metric 1 Role 0
I (6377499) LoRaMesher: 1 - 3188 via 3188 metric 1 Role 1
I (6377505) LoRaMesher: 2 - C6C4 via C6C4 metric 1 Role 1
I (6377511) LoRaMesher: 3 - 1428 via 1428 metric 1 Role 0
I (6377517) LoRaMesher: 4 - 7AFC via 7AFC metric 1 Role 0
I (6377523) LoRaMesher: 5 - E7A8 via E7A8 metric 1 Role 1
I (6377529) LoRaMesher: 6 - 1DB8 via 1DB8 metric 1 Role 0
I (6377535) LoRaMesher: 7 - 2598 via 2598 metric 1 Role 0
I (6377541) LoRaMesher: 8 - 50B8 via 50B8 metric 1 Role 0
I (6377547) LoRaMesher: 9 - 869C via 869C metric 1 Role 0
I (6377553) LoRaMesher: 10 - 148C via 148C metric 1 Role 0
I (6377559) LoRaMesher: 11 - 66DC via 66DC metric 1 Role 0
I (6377566) LoRaMesher: 12 - 7984 via 7984 metric 1 Role 0

```

Fig. 10. Routing table snapshot of gateway node DD3Ch.

node DD3C during the experiment. It contains the other 13 nodes of the network. Furthermore, the nodes 3188, C6C4, and E7A8 are indicated with role value 1 as a gateway.

The monitoring application on each of the 14 nodes periodically publishes data messages to the MQTT broker in the WiFi network. From the message data obtained at the MQTT broker, we filter out the values of the MessageId, Routing table size, and TxQ.

The messageId introduced in Section 5.3 is part of the application message header. Its value represents a mod256 counter which increases by one with every message sent from the application. The metric allows to verify if all messages of a node are received at the MQTT broker. The size of the routing table corresponds to the number of nodes in the network. This metric is obtained by a function call to the LoRaMesher library. The value is sent as part of the application message payload. The metric helps to understand if the routing table at each node is correctly maintained. The TxQ metric indicates the size of the send task queue. The value is obtained from the LoRaMesher library and it is also sent as part of the application message payload. The metric gives information about messages pending to be sent in reliable transfer operations.

The experiment is carried out with the following settings: SF 7 and a maximum LoRa packet size of 100 bytes. The routing packets are sent every 300 s. Each node sends the monitoring data every 120 s. The reliable message transfer mode of LoRaMesher is used. Hence, being the gateway nodes destination for the non-gateway nodes, the gateways interact with the sender nodes and acknowledge the reception of messages to the sender nodes.

In Figure 11 the evolution of the MessageId value is shown for the 14 nodes. A periodic pattern of the value can be observed for all nodes. It repeats about every 8,3 hours which corresponds to the configured periodicity with sending the counter value of increasing by one up to 256 every 2 minutes. A linear increase is seen for the messageId of the four gateway nodes. This is explained by the fact that gateway nodes sent the monitoring data over WiFi without loss of messages to the MQTT broker. A not completely smooth increase of the messageId value can be observed for non-gateway nodes. This is because, in our scenario, several nodes send data and routing packets that may overlap in time, and collisions in the LoRa mesh network happen which leads to packet losses. Since the reliable messaging mode re-sends the lost packets, some messages of the non-gateway nodes are received at the MQTT broker out-of-order. This observation is consistent with the experiments presented in the Sections 6.2, 6.3, and 6.4. We can observe that the behavior of this experiment shows a stable operation of the monitoring application on each node during the 24-hour duration.

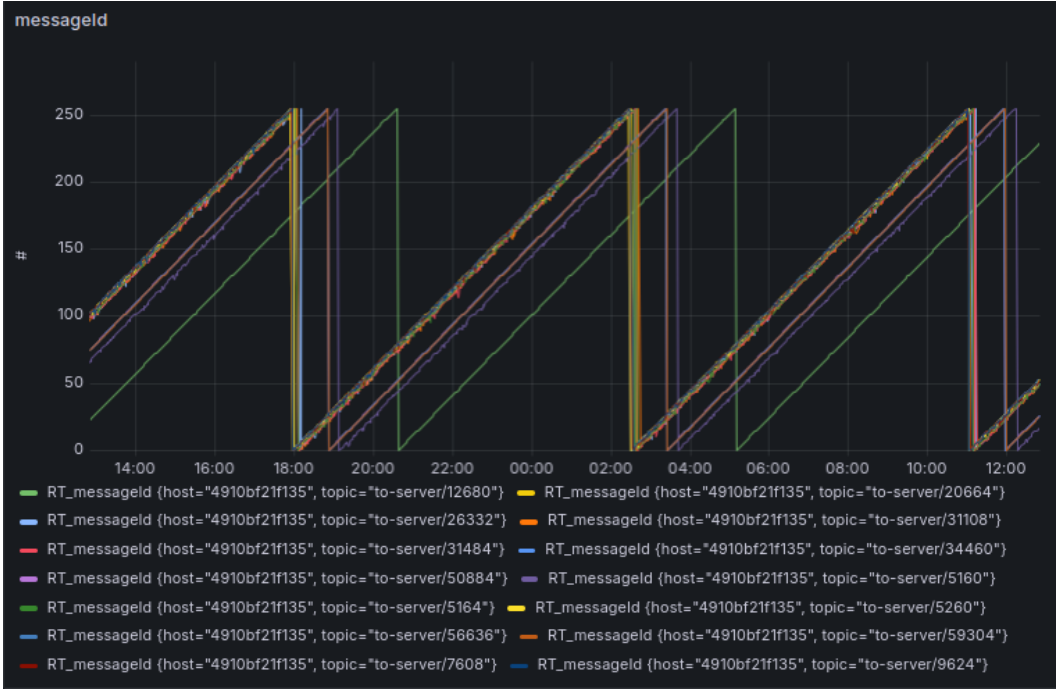


Fig. 11. Evolution of the value of the messageId field along a 24-hour experiment with 14 nodes.

In Figure 12 the Routing table size value is shown. It can be observed that for a few nodes, the value of 13, which corresponds to the other 13 nodes in the network (see also Figure 10), changed temporarily to 12 nodes. This indicates that collisions happened in the routing packets and occasionally a node deleted another node in its table when the information about that node was not refreshed before the timeout. Since the experiment had four gateway nodes, in case the deleted node in the routing table was a gateway node, the non-gateway node could still choose from the other three gateway nodes available in its routing table. Overall, the observed behavior is correct and shows how the routing table is maintained by all nodes under packet collisions that can happen in a real environment.

We analyze the TxQ value in Figure 13. The size of the send packet queue with up to five packets can be seen in the gateway nodes. This can be explained by the role of the gateway nodes, which as the destination of the ten non-gateway nodes, participate in reliable message delivery with each sender. Pending confirmation messages can increase the TxQ value while the send task whose priority is lower than that of the receive task is unable to clear its queue (Figure 5). This observation is compatible with the not completely smooth evolution of the messageId value of non-gateway nodes observed in Figure 11 where the resending of packets took place due to the effects of packet losses by collisions.

7 Discussion

The proposed middleware covers a gap in the availability of tools to enable applications on LoRa IoT nodes and hence can facilitate a broader use of LoRa mesh network as a communication substrate. As several authors pointed out in their works, such as [14] and [30], LoRa networks could be a

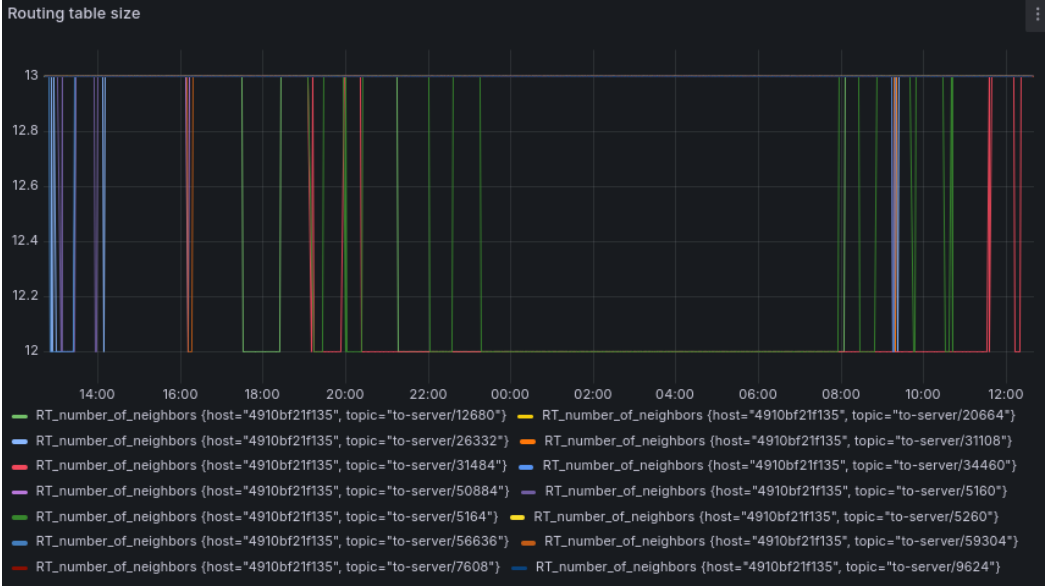


Fig. 12. Evolution of the routing table size value along a 24 hour experiment with 14 nodes.

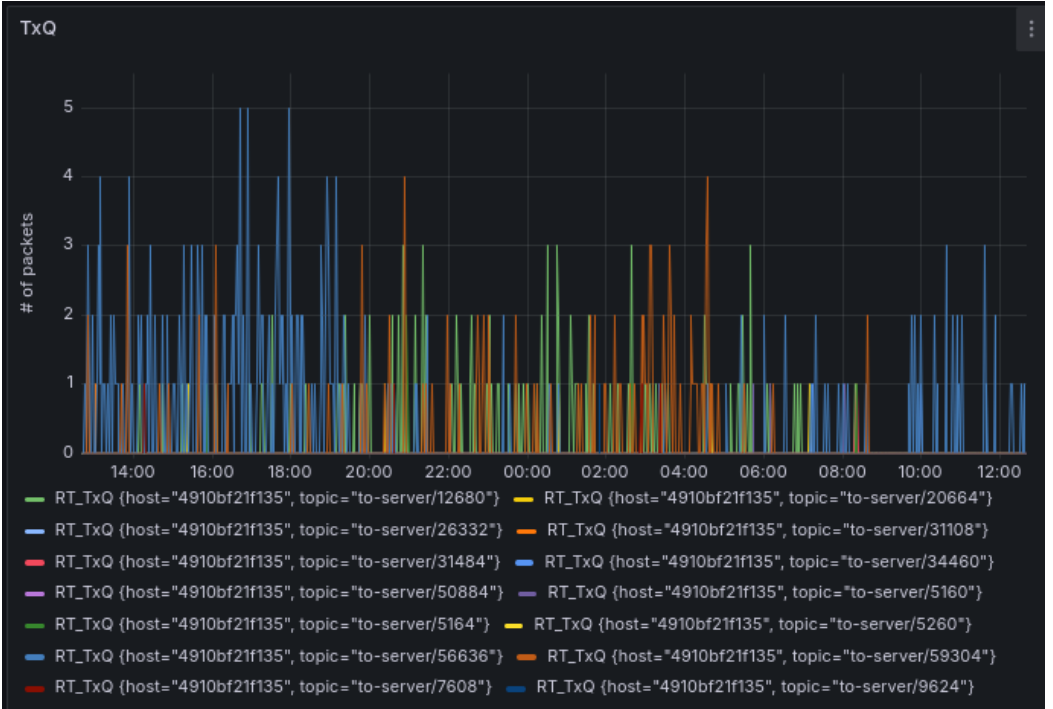


Fig. 13. Evolution of the TxQ value along a 24 hour experiment with 14 nodes.

means to connect the unconnected while other networks are not yet in place.³ Compared with other options, a LoRa network is a low-data rate network, but it is also very affordable worldwide since the hardware is cheap, open-source software is available, and the network does not need any license.

Another aspect that can push LoRa mesh networks and their applications as an important innovation for low-cost connectivity is the ease of deployment, which can also be done very quickly in comparison to other more complex types of networks. In addition, if the operation of the network protocol on the microcontroller board is stable and robust, and knowledge to operate the network is open and available to everybody, then there is a potentially low cost of maintenance. From this perspective, LoRa mesh networks and their applications can resemble the purpose of community networks [7], another approach to connect the unconnected, where in comparison we can position LoRa mesh networks as a technically less performant solution, covering only certain application cases, but being easier to deploy and maintain.

For improved usability of LoRa mesh networks, we consider the option for reliable data transfer, available not only in the middleware we propose, as explained in Section 4.2, but also presented in the work of others [5, 9, 24], a relevant feature. With reliable messaging and supporting large payloads, applications that do not well tolerate losses, such as text messaging, as pointed out in [30], can be implemented with LoRa communication while providing certain guarantees to the user. The cost of reliable packet delivery, however, is an increased traffic due to the additional control messages. In low-traffic networks, this may not affect performance, but in networks with higher traffic, the effect of the increasing number of collisions may need to be carefully assessed. Out-of-order reception of packets, such as shown in Section 6.4 in the experiment with two applications running concurrently, can be expected. Furthermore, a large number of pending data flows from one or more applications using reliable messaging can increase the send queue of a gateway node, as could be observed in the TxQ evolution of Figure 13 in Section 6.5. This can lead to scalability problems specifically in the gateway nodes due to limited hardware resources such as the memory of the microcontrollers. A balanced selection between gateway nodes by the other nodes of the LoRa mesh network could more evenly distribute the load on the available gateways, but currently, a gateway selection mechanism is not implemented in LoRaMesher.

To be able to distinguish between different applications, the LoRaMesher packet header was extended by 3 bytes, as described in Section 5.3, reducing thus the maximum payload size to 208 bytes. As a consequence, if a large amount of data has to be sent, the reduction of the payload size by the application identifier will result in a slight increase in the total number of packets sent. It can be remarked in this context that the identification of applications is accomplished, but the authentication of applications is not implemented. While authentication could be achieved by integrating existing cryptographic libraries, the number of additional bytes of the signature would reduce the amount of additional data that could be sent in that LoRa packet. In summary, both reliable message transfer and the capacity of running multiple applications increase the bandwidth utilization, the first by adding control messages and the latter by slightly increasing the LoRaMesher packet header.

LoRa mesh networks, compared to the mature LoRaWAN standard, have a much smaller body of research works very few open implementations, and lack practical experience. That impedes visibility as a potential solution for situations where LoRaWAN may be less suitable, for instance in deployments covering large geographic regions where the needed density of gateways may be costly or unfeasible. Also, the use of LoRa nodes as sensor nodes that are only sporadically operative to measure and send sensor values is a very present and dominating concept. Differently,

³IEEE Connecting the Unconnected Challenge. <https://ctu.ieee.org>

with the tendency to move machine learning to the edge and into microcontroller boards, there is a shift from sensor boards transmitting only raw sensor values to conducting on-board classification and prediction tasks while being permanently operational [17]. The middleware that we propose in this work can contribute to supporting networked machine-learning applications in LoRa mesh networks.

The LoRaMesher and LoRaChat software that is presented in this work has been made available as open-source in public repositories from the outset. LoRaMesher has gained the recognition of the community given that its git repository has been awarded more than 190 stars by today. The LoRaChat repository, which is more recent as it builds the middleware on top of LoRaMesher, is the layer that extends the basic usage of LoRaMesher, from a LoRa mesh network for a single application into a network service that can be used by multiple distributed applications. The LoRaMesher and LoRaChat repositories are provided with guidelines, which enable the community to reproduce the examples and use these examples as blueprints to build their applications. While we can consider the usability of LoRaMesher and LoRaChat as well achieved, the extension or customization of the software internals by others may require a significant initial effort. The design of the LoRaChat middleware, which is described in Section 5, allowed achieving important technical features but is complex to understand in the depth that allows doing internal modifications. LoRaMesher, explained in detail in [28], has received some contributions from the community, and with the deployment of initial demonstrators that use the middleware,⁴ we aim to raise the interest of the community in LoRaChat and gain developers to contribute to the effort.

8 Conclusion

This article presented a middleware to run distributed applications in a LoRa mesh network. For this, we extended the packet header of the LoRaMesher library with application-related information for the middleware to be able to manage multiple applications. Furthermore, we introduced roles for nodes. A gateway node, disseminated through routing table exchanges, provides any node within the LoRa mesh network with the information needed to send data to the Internet. From the architectural perspective of the software, we introduced the MessageManager as a central component of the design. The instances of applications and services are registered at the MessageManager at the node initialization, enabling the MessageManager process to be used on top of the network level for the message to be sent from the node's applications, or for demultiplexing received messages in the context of multiple applications.

We have implemented the middleware as an extension of the LoRaMesher library. In experiments with real nodes, we focus on distributed applications that span from the LoRa mesh network nodes to the Internet over MQTT communication. We demonstrate the bidirectional communication of applications between the LoRa mesh network and an Internet host. We also experimented with the case of running two distributed applications concurrently at the nodes and demonstrated the capacity of the middleware to support such a scenario. Furthermore, we ran a 24-hour experiment with 14 nodes to demonstrate the operation of the software on a larger scale and the stability of the implementation.

The LoRaMesher library and the LoRaChat middleware are open source and are available in public git repositories. They are provided with guidelines, examples, and applications. The community is supported to reuse and build upon the work.

In our next steps, we aim to understand the concept of services that can be provided by applications on nodes in LoRa mesh networks through MQTT communication and their integration into Internet-based applications. The possibility to make queries from the Internet to LoRa mesh

⁴<https://tomir.ac.upc.edu/loraupc/index.php>

network nodes is new in comparison to how sensor nodes in LoRa networks are traditionally used. It could be interesting to identify common building blocks used by our example applications to define the architecture of this type of application, as well as their performance. Given that capacity for cross-network communication of applications, a concrete applications area could be edge AI, where the middleware could enable networked machine learning applications that are provided as a service by tiny IoT nodes.

References

- [1] [n. d.]. Meshtastic: Open Source hiking, pilot, skiing and secure GPS mesh communicator. Retrieved October 20, 2022 from <https://meshtastic.org/>
- [2] [n. d.]. NeoCortec home page. Retrieved December 01, 2024 from <https://neocortec.com/>
- [3] [n. d.]. Orama-System home page. Retrieved December 01, 2024 from <https://orama-system.com/>
- [4] [n. d.]. RadioLib: Universal wireless communication library for embedded devices. Retrieved December 28, 2024 from <https://github.com/jgromes/RadioLib>
- [5] Benjamín Arratia, Erika Rosas, Carlos T. Calafate, Juan-Carlos Cano, José M. Cecilia, and Pietro Manzoni. 2024. AlloRa: Empowering environmental intelligence through an advanced LoRa-based IoT solution. *Computer Communications* 218 (2024), 44–58. DOI : <https://doi.org/10.1016/j.comcom.2024.02.014>
- [6] Benjamin Arratia, Erika Rosas, Ermanno Pietrosemoli, Marco Zennaro, José Maria Cecilia, and Pietro Manzoni. 2024. AI*LoRa: Enabling efficient long-range communication with machine learning at the edge. In *Proceedings of the 25th International Symposium on Theory, Algorithmic Foundations, and Protocol Design for Mobile Networks and Mobile Computing (MobiHoc '24)*. Association for Computing Machinery, New York, NY, USA, 458–463. DOI : <https://doi.org/10.1145/3641512.3690040>
- [7] Roger Baig, Ramon Roca, Felix Freitag, and Leandro Navarro. 2015. guifi.net, a crowdsourced network infrastructure held in common. *Computer Networks* 90 (2015), 150–165. DOI : <https://doi.org/10.1016/j.comnet.2015.07.009> Crowdsourcing.
- [8] Riccardo Berto, Paolo Napoletano, and Marco Savi. 2021. A LoRa-based mesh network for peer-to-peer long-range communication. *Sensors* 21, 13 (2021). DOI : <https://doi.org/10.3390/s21134314>
- [9] Angelica Moreno Cardenas, Miguel Kiyoshi Nakamura Pinto, Ermanno Pietrosemoli, Marco Zennaro, Marco Rainone, and Pietro Manzoni. 2020. A low-cost and low-power messaging system based on the LoRa wireless technology. *Mobile Networks and Applications* 25 (2020), 961–968.
- [10] Roger Pueyo Centelles, Roc Mesequer, Felix Freitag, Leandro Navarro, Sergio F. Ochoa, and Rodrigo M. Santos. 2021. LoRaMoto: A communication system to provide safety awareness among civilians after an earthquake. *Future Generation Computer Systems* 115 (2021), 150–170. DOI : <https://doi.org/10.1016/j.future.2020.07.040>
- [11] Reinaldo Cotrim, Flávio Assis, Alexsandro dos Santos Brito, Yslai Silva Peixoto, and Leandro Santos Peixoto. 2024. Multi-Hop LoRa-based underground network for monitoring soil moisture in agriculture. *Computers and Electronics in Agriculture* 227 (2024), 109592. DOI : <https://doi.org/10.1016/j.compag.2024.109592>
- [12] Mauro A. A. da Cruz, Joel José P. C. Rodrigues, Jalal Al-Muhtadi, Valery V. Korotaev, and Victor Hugo C. de Albuquerque. 2018. A reference model for internet of things middleware. *IEEE Internet of Things Journal* 5, 2 (2018), 871–883.
- [13] Christian Ebi, Fabian Schaltegger, Andreas Rüst, and Frank Blumensaat. 2019. Synchronous LoRa mesh network to monitor processes in underground infrastructure. *IEEE Access* 7 (2019), 57663–57677. DOI : [10.1109/ACCESS.2019.2913985](https://doi.org/10.1109/ACCESS.2019.2913985)
- [14] Atonu Ghosh, Sudip Misra, Venkanna Udutalapally, and Debanjan Das. 2023. LoRaute: Routing messages in backhaul LoRa networks for underserved regions. *IEEE Internet of Things Journal* 10, 22 (2023), 19964–19971. DOI : <https://doi.org/10.1109/JIOT.2023.3281941>
- [15] Martin Haubro, Charalampos Orfanidis, George Oikonomou, and Xenofon Fafoutis. 2020. TSCH-over-LoRa: Long range and reliable IPv6 multi-hop networks for the internet of things. *Internet Technology Letters* 3, 4 (2020), e165. DOI : <https://doi.org/10.1002/itl2.165>
- [16] Jonas Höchst, Lars Baumgärtner, Franz Kuntke, Alvar Penning, Artur Sterz, Markus Sommer, and Bernd Freisleben. 2023. Mobile device-to-device communication for crisis scenarios using low-cost LoRa modems. In *Proceedings of the Disaster Management and Information Technology: Professional Response and Recovery Management in the Age of Disasters*. Springer, 235–268.
- [17] Rakhee Kallimani, Krishna Pai, Prasoon Raghuvanshi, Sridhar Iyer, and Onel L. A. López. 2024. TinyML: Tools, applications, challenges, and future research directions. *Multimedia Tools and Applications* 83, 10 (2024), 29015–29045. DOI : <https://doi.org/10.1007/s11042-023-16740-9>
- [18] Nils Karges, Jeroen Staab, Jürgen Rauh, Martin Wegmann, and Hannes Taubenböck. 2022. Soundscapes on edge - the real-time machine learning approach for measuring soundscapes on resource-constrained devices. In *Proceedings of the 24th International Congress on Acoustics*. 128–139. Retrieved from <https://elib.dlr.de/190129/>

- [19] Chenning Li and Zhichao Cao. 2023. LoRa networking techniques for large-scale and long-term IoT: A down-to-top survey. *ACM Computing Surveys* 55, 3 (2023), 1–36. DOI : <https://doi.org/10.1145/3494673>
- [20] Nil Llisterri Giménez, Joan Miquel Solé, and Felix Freitag. 2023. Embedded federated learning over a LoRa mesh network. *Pervasive and Mobile Computing* 93 (2023), 101819. DOI : <https://doi.org/10.1016/j.pmcj.2023.101819>
- [21] Mike McCauley. 2014. *RadioHead Packet Radio Library for Embedded Microprocessors*. Retrieved from <https://www.airspayce.com/mikem/arduino/RadioHead/>
- [22] Rodolfo Medeiros, Silvio Fernandes, and Paulo G. G. Queiroz. 2022. Middleware for the internet of things: A systematic literature review. *JUCS: Journal of Universal Computer Science* 28, 1 (2022).
- [23] Christos Milarokostas, Dimitris Tsolkas, Nikos Passas, and Lazaros Merakos. 2023. A comprehensive study on LPWANs with a focus on the potential of LoRa/LoRaWAN systems. *IEEE Communications Surveys and Tutorials* 25, 1 (2023), 825–867. DOI : <https://doi.org/10.1109/COMST.2022.3229846>
- [24] Kiyoshi Nakamura, Pietro Manzoni, Alessandro Redondi, Edoardo Longo, Marco Zennaro, Juan-Carlos Cano, and Carlos T. Calafate. 2022. A LoRa-based protocol for connecting IoT edge computing nodes to provide small-data-based services. *Digital Communications and Networks* 8, 3 (2022), 257–266. DOI : <https://doi.org/10.1016/j.dcan.2021.08.007>
- [25] Roger Pueyo Centelles. 2021. *Towards LoRa Mesh Networks for the IoT*. Ph. D. Dissertation. Universitat Politècnica de Catalunya, Computer Architecture Department, Barcelona, Spain.
- [26] Roger Pueyo Centelles, Felix Freitag, Roc Meseguer, and Leandro Navarro. 2021. Beyond the star of stars: An introduction to multihop and mesh for LoRa and LoRaWAN. *IEEE Pervasive Computing* 20, 2 (2021), 63–72. DOI : <https://doi.org/10.1109/MPRV.2021.3063443>
- [27] Jumpei Sakamoto, Daiki Nobayashi, Kazuya Tsukamoto, Takeshi Ikenaga, Goshi Sato, and Kenichi Takizawa. 2023. Improvement of TCP performance based on characteristics of private LoRa interface. In *Proceedings of the Annual Computers, Software, and Applications Conference (COMPSAC)*. 998–999. DOI : <https://doi.org/10.1109/COMPSAC57700.2023.00145>
- [28] Joan Miquel Solé, Roger Pueyo Centelles, Felix Freitag, and Roc Meseguer. 2022. Implementation of a LoRa mesh library. *IEEE Access* 10 (2022), 113158–113171. DOI : <https://doi.org/10.1109/ACCESS.2022.3217215>
- [29] Zehua Sun, Huanqi Yang, Kai Liu, Zhimeng Yin, Zhenjiang Li, and Weitao Xu. 2022. Recent advances in LoRa: A comprehensive survey. *ACM Transactions on Sensor Networks* 18, 4 (2022), 44 pages. DOI : <https://doi.org/10.1145/3543856>
- [30] Morgan Vigil-Hayes, Md Nazmul Hossain, Alexander K. Elliott, Elizabeth M. Belding, and Ellen Zegura. 2022. LoRaX: Repurposing LoRa as a low data rate messaging system to extend internet boundaries. In *Proceedings of the ACM SIG-CAS/SIGCHI Conference on Computing and Sustainable Societies (COMPASS) (COMPASS '22)*. Association for Computing Machinery, New York, NY, USA, 195–213. DOI : <https://doi.org/10.1145/3530190.3534807>
- [31] Andrew Wei-Loong Wong, Say Leng Goh, Mohammad Kamrul Hasan, and Salmah Fattah. 2024. Multi-hop and mesh for LoRa networks: Recent advancements, issues, and recommended applications. *ACM Computing Surveys* 56, 6, Article 136 (2024), 43 pages. DOI : <https://doi.org/10.1145/3638241>

Received 7 February 2024; revised 30 May 2025; accepted 25 June 2025